ET AI HACKATHON 2026

Zero-Harm

AI-Powered Industrial Safety Intelligence for Zero-Harm Operations

Problem Statement 1 — Industrial Intelligence / Worker Safety / Geospatial Safety Analytics

Technical Documentation & Solution Architecture

Working Prototype · Architecture · Measured Results · Regulatory Mapping · Evaluation Methodology

Contents

1. Executive Summary

Zero-Harm is a working prototype of a compound risk detection platform for heavy industrial plants. It directly answers the brief's core diagnosis: Indian industrial facilities are not short of sensors, permits, or maintenance systems — they are short of an intelligence layer that fuses those independent data sources into a single, real-time risk picture and acts on it before an incident, not after.

The prototype ingests four independent data streams — gas sensor readings, active work permits, maintenance activity, and worker location — and correlates them using a rule-based fusion engine to detect dangerous combinations no single system would flag alone. The flagship example, modelled directly on the January 2025 Visakhapatnam Steel Plant coke oven incident described in the brief, is the co-occurrence of a hot-work permit and rising gas levels in the same zone during active maintenance.

The deliverable includes a FastAPI backend implementing the Compound Risk Detection Engine and a live geospatial dashboard (the Geospatial Safety Heatmap deliverable) that visualises zone-level risk in real time, lists explainable alerts with recommended actions, and lets a judge trigger the incident scenario on demand to see detection happen live.

New in this version: An Isolation Forest anomaly detection layer now runs alongside the rule engine as a fifth signal (Section 5.5), catching gas behaviour that deviates from a zone's own historical pattern even before a fixed threshold is crossed. Section 11 adds measured, reproducible numbers comparing the compound engine against a single-sensor baseline, and Section 12 adds a direct mapping between every implemented feature and the specific Indian safety regulation it satisfies.

2. Problem Context

India's heavy industrial sector records thousands of fatal workplace accidents annually, and DGFASLI figures cited in the brief put FY2023 fatalities above 6,500 — excluding most mining and construction. The January 2025 Visakhapatnam Steel Plant coke oven battery explosion is the reference incident for this problem statement: gas pressure sensors detected the danger, but no intelligence layer connected those readings to an operational decision in time, despite the facility having functioning gas detectors, permit-to-work controls, and SCADA.

A 2024 FICCI survey referenced in the brief found that over 60% of large industrial facilities still coordinate between their own digital safety tools through manual handoffs. The technology exists in silos; what is missing is fusion, correlation, and pre-emptive action — exactly the gap this prototype is built to close.

3. Solution Overview

The prototype implements two of the six illustrative components from the problem brief end-to-end, and is architected so the remaining four can be added as independent services without changing the core fusion logic.

Implemented in this prototype

- Compound Risk Detection Engine — multi-source correlation across gas sensors, permits, maintenance activity, and worker location, producing a weighted 0–100 risk score per zone with explainable triggers.
- AI Anomaly Detection Layer (NEW) — an Isolation Forest model per zone adds a fifth, machine-learned signal that flags gas behaviour deviating from that zone's own historical pattern, independent of the fixed-threshold rules.
- Geospatial Safety Heatmap — live plant-layout visualisation showing risk state per zone, updating every 3 seconds, with zone-level drill-down.

Designed-for extension points

- Digital Permit Intelligence Agent — the permit/gas correlation logic already implemented is the core of this component; extending it to cross-check simultaneous operations across all permit types is a configuration change, not a redesign.
- Incident Pattern Intelligence — a RAG layer over historical near-miss and OISD/Factory Act guidance can subscribe to the same alert stream and add regulatory context.
- Emergency Response Orchestrator — the alert payload already contains a recommended action; wiring it to evacuation/notification channels is an integration task against the existing /api/alerts endpoint.
- Quality & Compliance Audit Agent — the permit and maintenance data model already carries the fields (zone, status, timestamps) an audit/compliance agent would need to cross-reference against OISD/DGMS/Factory Act requirements.

4. System Architecture

The platform follows a four-layer pipeline: heterogeneous data sources, an ingestion/normalisation layer, a multi-agent fusion engine, and action/intelligence outputs.

Layer 1 — Data Sources

IoT Gas Sensors, SCADA Systems, Permit-to-Work, CMMS/Maintenance, CCTV Feeds, Worker Location (RFID/BLE badges).

Ingestion & Normalisation Layer

OPC-UA / MQTT / REST connectors feeding a unified event schema.

Layer 2 — Compound Risk Detection Engine (multi-agent fusion)

Gas Agent, Permit Agent, Maintenance Agent, Worker Density Agent, and a designed-for Vision/CCTV Agent feed into a single Fusion & Compound Risk Scoring stage that produces a 0–100 score mapped to LOW / MODERATE / HIGH / CRITICAL.

Layer 3 — Action & Intelligence Outputs

Geospatial Risk Heatmap, Compound Risk Alerts, and two designed-for extensions: Emergency Response and Compliance Audit Trail.

In production, Layer 1 sources would connect via OPC-UA/MQTT (SCADA, gas sensors), REST/DB connectors (permit-to-work and CMMS systems), and RFID/BLE/UWB feeds (worker location) rather than the in-memory simulator used for the hackathon demo. No other layer needs to change to go live — the simulator was built to match the exact data shapes a real integration would produce.

5. Core Components

5.1 Data Simulator (data_simulator.py)

Models a six-zone plant layout (coke oven battery, gas cleaning plant, blast furnace bay, tar tank farm, sinter plant, rolling mill) matching the hazard classes referenced in the brief. Generates realistic gas sensor drift, seeds active permits and maintenance jobs, and exposes a trigger_incident() function that reproduces the compound failure pattern from the reference incident on demand.

5.2 Compound Risk Detection Engine (risk_engine.py)

Implements the fusion logic as four independent 'agent' functions — one per data source — whose outputs are combined by a single assess_zone() orchestration function. Each agent can be swapped for a trained ML model later without touching the fusion or scoring logic.

Agent	Inspects	Detects
Gas Agent	Live sensor readings vs. per-gas warning/critical thresholds	Elevated or critical gas concentration by zone
Permit Agent	Active permits by type and zone	Hot-work or confined-space permits active during a hazard condition
Maintenance Agent	Active CMMS/maintenance jobs by zone	Maintenance activity co-occurring with a gas rise
Worker Density Agent	Worker badge locations by zone	Headcount exposed to an elevated-risk zone

5.5 AI Anomaly Detection Layer (anomaly_detector.py) — NEW

The four rule-based agents above catch KNOWN dangerous combinations — a permit issued during elevated gas, maintenance during a gas rise, and so on. They cannot catch a pattern nobody thought to write a rule for. The anomaly detection layer closes that gap: one Isolation Forest model is trained per zone on that zone's own historical gas behaviour (value and rate-of-change), and flags readings that deviate from the zone's normal pattern even before any fixed threshold is crossed.

This is deliberately additive, not a replacement: assess_zone() takes the anomaly result as one more optional signal, contributing +15 to the compound score with its own explainable trigger line — consistent with the project's explainability principle that no signal is ever a bare number without a stated reason.

Why Isolation Forest specifically: it is unsupervised (no labelled incident data required, which real plants rarely have in volume), trains in milliseconds per zone, and produces a continuous confidence score rather than a black-box binary flag.

Tuning note: the detection threshold is calibrated conservatively (65% abnormality confidence) so that routine baseline sensor noise does not trigger false alarms — verified directly: all six zones remain silent at baseline, and the signal only activates once a genuine sustained gas ramp begins.

5.3 API Layer (main.py)

A FastAPI service exposing the fused plant state and live alerts as REST endpoints, plus a background task that advances the simulation every two seconds so the dashboard feels live.

5.4 Geospatial Safety Heatmap (frontend/index.html)

A SCADA-styled single-page dashboard: a zone map coloured by live risk level, an explainable alert feed (each alert lists the specific triggers and a recommended action, not just a score), and permit/sensor tables. Includes a 'Simulate Compound Incident' control so judges can trigger the detection scenario live during a demo.

Two additional panels (NEW) complete the evaluation story on-screen: a live 'Evaluation Metrics' panel showing Detection Rate, False Negatives, Lead Time, and AI Anomaly Confidence as KPI cards fed directly from GET /api/benchmark; and a distinctly-styled 'Estimated Business Impact' panel — visually marked with a dashed border and an ILLUSTRATIVE ESTIMATE badge — so the projected figures are never confused with the measured detection metrics beside them.

6. Compound Risk Scoring Methodology

Each contributing signal adds a calibrated weight to a zone's compound risk score (capped at 100).

Signal	Weight	Rationale
Gas reading — WARNING	+25	Early indicator, not yet dangerous alone
Gas reading — CRITICAL	+45	Immediate hazard on its own
Hot work permit + elevated gas	+30	Classic ignition risk — the Vizag incident pattern
Confined space entry + elevated gas	+30	Asphyxiation / toxic exposure risk
Maintenance activity + elevated gas	+20	Unplanned intervention during a hazard window
≥2 overlapping permits in one zone	+10	Coordination risk between concurrent crews
≥3 workers present + elevated gas	+10	Exposure magnitude
AI Anomaly Detector flags deviation (NEW)	+15	Isolation Forest signal — catches abnormal gas behaviour independent of fixed thresholds

Score bands map to action levels: 0–19 LOW (routine monitoring), 20–44 MODERATE (increase monitoring, notify supervisor), 45–69 HIGH (suspend relevant permits, notify safety officer), 70–100 CRITICAL (evacuate zone, suspend all active permits, dispatch emergency response) — directly operationalising the brief's requirement to 'trigger preemptive interventions before they escalate.'

7. Data Model

Entity	Key fields	Source system in production
Zone	zone_id, name, x/y coordinates, hazard_class	Plant layout / GIS master data
GasReading	sensor_id, zone_id, gas_type, value_ppm, thresholds	IoT gas sensors / SCADA historian
Permit	permit_id, permit_type, zone_id, issued_to, validity, status	Digital permit-to-work system
MaintenanceActivity	activity_id, zone_id, description, active flag	CMMS (e.g. SAP PM, Maximo)
WorkerLocation	worker_id, name, zone_id, timestamp	RFID / BLE / UWB badge system
RiskAlert	zone_id, risk_level, risk_score, triggers[], recommended_action	Generated by the fusion engine

8. Technology Stack

Layer	Technology	Notes
Backend API	Python 3.12, FastAPI, Pydantic v2	Async, typed, autogenerates OpenAPI docs at /docs
AI Anomaly Detection	scikit-learn (Isolation Forest), NumPy	One model per zone, trained at startup, millisecond inference
Simulation	In-memory Python state + asyncio background task	Zero external dependencies for the hackathon demo
Frontend	Vanilla HTML/CSS/JS, SVG for the geospatial map	No build step — runs from static files served by FastAPI
Server	Uvicorn (ASGI)	Single command to run: uvicorn main:app
Suggested production additions	OPC-UA/MQTT client, PostgreSQL + PostGIS, graph DB, RAG stack	Not required for the current prototype

9. API Reference

Endpoint	Method	Description
/api/dashboard	GET	Single fused payload: zones, sensors, permits, maintenance, workers, alerts
/api/zones	GET	Plant zone master data
/api/sensors	GET	Current gas sensor readings
/api/permits	GET	Active and historical permits
/api/maintenance	GET	Maintenance/CMMS activity
/api/workers	GET	Current worker locations
/api/alerts	GET	Current compound risk alerts, highest score first
/api/simulate/incident/{zone_id}	POST	Triggers the demo compound-risk scenario in a zone
/api/simulate/clear/{zone_id}	POST	Resets a zone to baseline
/api/benchmark	GET	Live precision/recall/F1 metrics, lead-time stats, and business impact estimate

10. Running the Prototype & Demo Script

Setup

- cd backend && pip install -r requirements.txt
- uvicorn main:app --reload --port 8000
- Open http://localhost:8000 in a browser

3-minute demo script

- Show the live dashboard at baseline — all zones green, sensor table nominal.
- Select 'Coke Oven Battery 3' and click 'Simulate Compound Incident' — this reproduces the Vizag pattern: gas levels climb, an emergency maintenance job starts, and a hot-work permit is issued in the same zone.
- Point out the zone marker turning amber then red, and the new alert appearing with its specific triggers and recommended action.
- Open the permits and sensor tables to show the underlying data the engine correlated.
- Click 'Clear Incident' to reset, and close with the extension points and measured results below.

11. Measured Results — Compound Engine vs. Single-Sensor Baseline

The brief's evaluation focus explicitly names 'compound risk detection accuracy vs. single-sensor baselines' and 'reduction in false negative rate.' Rather than asserting this qualitatively, we built a standalone, reproducible comparison script (baseline_vs_compound_demo.py, included in the GitHub repository) that runs six realistic zone timelines — four genuinely dangerous compound scenarios and two harmless control scenarios — through both a single-sensor baseline (alarm only when gas crosses the CRITICAL threshold) and our compound engine (alarm when the fused score crosses the HIGH action band).

Result Summary

Metric	Single-Sensor Baseline	Zero-Harm Compound Engine
Dangerous scenarios caught	3 of 4	4 of 4

False negatives (missed real danger)	1	0
Average early-warning lead time	— (reference point)	15 minutes earlier
False alarms on harmless scenarios	0	0

The missed scenario is instructive: a hot-work permit issued in a zone where gas rose steadily but never crossed the CRITICAL threshold on its own. A single-sensor system would have stayed silent all day. The compound engine flagged it as HIGH risk the moment the permit was issued during an already-elevated gas reading — the exact class of silent risk described in the brief's problem context.

Reproducibility: Every judge can run this script themselves — python3 Zero-Harm_Baseline_Comparison_Script.py — with no external dependencies beyond scikit-learn/numpy (matplotlib optional, for charts). The scenario data is fully visible and editable in the source file, not a black box. The same numbers are also served live from the running prototype at GET /api/benchmark and rendered as KPI cards on the dashboard, so a judge never has to take a static document's word for it.

11.1 Precision / Recall / F1 / Accuracy

Computed from a real confusion matrix over the 6 main scenarios (4 dangerous = positive class, 2 harmless = negative class):

System	Precision	Recall	F1 Score	Accuracy
Single-Sensor Baseline	1.000	0.750	0.857	0.833
Zero-Harm Compound Engine	1.000	1.000	1.000	1.000

Neither system produced a false positive on the 2 harmless control scenarios, so precision is perfect for both — the compound engine's advantage is entirely in recall (catching every real danger), which is exactly the 'reduction in false negative rate' the brief asks for.

11.2 Business Impact Estimate — Methodology (Illustrative, Not Measured)

The dashboard's 'Estimated Business Impact' panel projects the measured recall improvement (0.75 → 1.00, a 25-percentage-point gain) onto one transparent, stated assumption: a mid-size plant experiencing approximately 40 similar compound-risk situations per year, each averaging 6 hours of avoidable downtime at ₹75,000/hour. This yields an illustrative ~10 additional incidents caught, ~60 hours of downtime avoided, and ~₹45 lakh in avoided cost per year.

This is explicitly labelled as an estimate in the dashboard UI itself, not presented as measured plant data — the assumptions are returned alongside every number from GET /api/benchmark so they can never be mistaken for real-world outcomes. Change the constants at the top of backend/benchmark.py to model a different plant size.

12. Regulatory Compliance Mapping

The brief names 'regulatory compliance coverage (OISD/Factory Act/DGMS)' as an explicit evaluation focus. The table below maps each implemented feature to the specific regulation it satisfies.

Feature	Regulation	In simple terms
Permit Agent	Factory Act 1948, Sec. 36 & 38	Checks continuously that risky work is authorised and safe — not just once at sign-off.
Gas Agent	OISD-STD-105 / OISD-STD-106	Meets the requirement for working gas detection, and additionally decides what action to take.
Compound Risk Alert	Factory Act 1948, Sec. 7A	Supports the occupier's duty to prevent harm proactively, not just record it afterward.
Maintenance Agent	DGMS Circular on Permit-to-Work	Automates the cross-check between maintenance

	Systems	and permit systems that DGMS requires.
Alert log (triggers + action)	Factory Act 1948, Sec. 87 / accident reporting rules	Every alert is already a timestamped, evidence-backed audit trail for inspectors.

Honest scope note: a full RAG-based reader that cross-references complete OISD/DGMS/Factory Act documents clause-by-clause is a Section 15 roadmap item, not built for the hackathon. What is built is the mapping between our existing implemented data fields (zone, permit type, timestamps, gas readings) and the specific clauses above — so this extension is a data-plumbing exercise, not a redesign.

13. Evaluation Focus — How the Prototype Addresses It

Evaluation Focus (from brief)	How the prototype addresses it
Compound risk detection accuracy vs. single-sensor baselines	Measured directly in Section 11: 4/4 vs 3/4 across six test scenarios, with a reproducible script.
Prediction lead time before incident threshold	Measured at 15 minutes average lead time in the same comparison (Section 11).
Geospatial evidence quality	The heatmap renders every zone's real-time state with drill-down to specific triggers, on a layout representative of the plant described in the brief.
Regulatory compliance coverage (OISD/Factory Act/DGMS)	Directly mapped feature-by-feature in Section 12, with an honest roadmap note on remaining scope.
Reduction in false negative rate	Zero false negatives in the compound engine vs. one in the baseline, across the same six scenarios (Section 11).

14. Judging Criteria Alignment

Criterion	Weight	How this submission addresses it
Innovation	25%	Multi-agent compound correlation rather than single-sensor alerting, now combined with an unsupervised Isolation Forest anomaly layer; every signal — rule-based or ML — surfaces as an explainable trigger, never an opaque score.
Business Impact	25%	Directly modelled on a real, named incident (Vizag, Jan 2025), the FICCI-reported systemic gap, and now backed by measured detection numbers and a legal compliance mapping.
Technical Excellence	20%	Clean separation of ingestion, fusion, and presentation layers; typed data contracts; swap-in path from simulated to real data sources without redesign.
Scalability	15%	Stateless FastAPI service; each agent and each zone assessment is independent, so horizontal scaling requires no architectural change.
User Experience	15%	Live, glanceable geospatial view for a safety officer, with explainable, action-oriented alerts rather than raw data dumps.

15. Roadmap to Production

- Replace the in-memory simulator with real OPC-UA/MQTT sensor feeds and a permit-to-work / CMMS API integration — no change needed to risk_engine.py's interface.
- Add the Incident Pattern Intelligence agent: a RAG layer over historical near-miss reports and OISD/Factory Act guidance, surfaced alongside each alert.
- Add the Emergency Response Orchestrator: wire /api/alerts to multi-channel notification (SMS/radio/PA) and an automated preliminary incident report generator.

- The Isolation Forest anomaly layer (Section 5.5) is trained on synthetic historical patterns for the hackathon demo; in production it retrains on each zone's actual historical SCADA log, and per-gas-type models can be added where multiple gas types share a zone.
- Add a knowledge graph (equipment–permit–risk relationships) to support the Quality & Compliance Audit Agent described in the brief.
- Add authentication, audit logging of every alert and operator action, and integration with the plant's existing SCADA HMI for a single pane of glass.

— End of document —